

NETSIPS PDF NEWSLETTER

2020-05-04

Contents

1. How to change the background color of the input field when it is selected?

Click here to read

2. Checking If One Array Contains an Item from Another Array in...

Click here to read

3. Nominal vs. Real GDP

Click here to read

4. How to hide your JavaScript code from View Source

Click here to read

5. Firebase authentication in Chrome Extension popup

Click here to read

6. Building a Chrome Extension with Firebase RTD

Click here to read

7. How North Korea uses bitcoin to get around US sanctions

Click here to read

8. Who is Vinay Dubey, the man that threatened a foot march from Mumbai to North India if train services weren't resumed?

Click here to read

How to change the background color of the input field when it is selected?

You should get the password reset instructions via email soon.
If you dont get the email, please check your spam folder.
If you need more help, please contact our support team.

Checking If One Array Contains an Item from Another Array in...

2019-07-20

You know what old school is? Old school is putting a for loop inside of a for loop to check to see if two arrays contain the same element.

const a1 = [1, 2, 3, 4, 5]; const a2 = [5, 6, 7, 8, 9];

This is a simply array example, but you could also assume that they are arrays of objects, and you want to check properties on those objects.

With the arrays above, how would you check to see if a2 contains a value that is in a1 ?

Okay, now how would you do it without two for loops?

This is where JavaScript\'s some() method comes into play. This allows you to test if "some" elements (by not necessarily all) in an array pass a certain test. So if we\'re trying to check elements in both arrays, what would that look like?

const b = a1.some((val) => a2.indexOf(val) !== -1);

In the example about, we use some() , which performs a test on each element in the a1 array. During that loop--which we conveniently have in a single line arrow function--we then check that the second array (a2) contains the value (val) from the first array by using the array object\'s indexOf() method. If the index value is not -1, then we know the array contains the value, and this line will return true. Since we\'re using some() , if at least one test is true, then "some" of the items are in each array, which then means that b is true.'

Nominal vs. Real GDP

GDP stands for gross domestic product and is the measure of the total economic output of the goods and services of a country. GDP is usually expressed on an annual basis, but is sometimes expressed on a quarterly basis within a year.Real GDP is equal to the economic output adjusted for the effects of inflation. Nominal GDP is economic output without the inflation adjustment. Nominal GDP is usually higher than real GDP because inflation is typically a positive number. Nominal GDP is used when comparing different quarters of output within the same year. When comparing the GDP of two or more years, real GDP is used because, by removing the effects of inflation, the comparison of the different years focuses solely on volume. The year-to-year comparison for real GDP requires what is called a base year. The base year is nothing more than the year to which all the other years are adjusted. Think of it as the as if year in that all the other years' GDP must be adjusted as if prices were the same in that year as they were in the base year. For example, Callowaynia is a small island country that only produces one product, golf balls. Callowaynia's nominal GDP was 70 million for 2012, and 90 million for 2013. $20 million seems like a substantial increase in GDP. But in 2012, the price of a golf ball was $1. In 2013, the price for a golf ball increased to $1.20. Callowaynias 2012 total output, in units, was 70 million golf balls ($70 million/$1). For 2013 the total output was 75 million golf balls ($90 million/$1.20). If the 2013 nominal GDP was adjusted for inflation so that the total units produced were calculated at the 2012 price of $1 (since 2012 is the base year in this example), then real GDP for 2013 is $75 million (75 million units X $1 per unit). With this adjustment, there is still an increase in GDP, but only $5 million, which is solely attributable to the increase in production from 70 million golf balls to 75 million golf balls."

How to hide your JavaScript code from View Source

By: Tam Hanna | 2019-03-22

If you don\'t take precautions with your JavaScript code, you\'re making life easy for anyone who wants to clone it. But if your programming processes are more than one click away, attackers will prefer to clone a competitor\'s software. Bots and other miscreants combing the internet for AWS or Azure credentials usually do not bother with obfuscated code the next website\'s login information can also be used to mine bitcoin and can be harvested with less effort.

Obfuscating JavaScript used to be complex. However, the process has become significantly easier during the last few years. Today, even small companies can and should protect their code from prying eyes. In some cases, getting secure is just an invocation of a Node.js package away.

The following steps introduce a commonly used JavaScript obfuscator, and also look at a few other topics related to the problem at hand.

Our JavaScript obfuscator lives in the Node runtime environment. Let us start out by checking the versions used. The output below provides the version state found on yours truly\'s workstation used for the following steps:

tamhan@tamhan-thinkpad:~$ node --version v8.12.0 tamhan@tamhan-thinkpad:~$ npm --version 6.4.1

Javascript-obfuscator should be installed into the global assembly cache of your workstation. Invoke npm with the -g parameter and don\'t forget to provide superuser rights the actual deployment process should be done in a few seconds.

tamhan@tamhan-thinkpad:~$ sudo npm install -g javascript-obfuscator [sudo] password for tamhan: . . . + javascript-obfuscator@0.18.1 added 103 packages from 162 contributors in 4.4s

Testing obfuscation works best if we have some \'real\' code. So let us start out with a small HTML webpage. It loads a JavaScript file called worker.js, declares a button and contains a small bit of inline scripting. When it\'s loaded in a browser, click the button to show a textbox.

<html> <body> <script src="worker.js"></script> <script> function worker(){ alert(doTheTrick()); } </script> <button type="button" onclick="worker( )">Click Me!</button> </body> </html>

Worker.js starts out with a string variable. They are a classic attack target if a ROM is to be decoded, the assembler usually starts out by looking for tables containing string sequences. Furthermore, encryption is performed using a set of variables with very \'speaking\' names.

var myString = "Hello from Future plc" function doTheTrick(){ var chiffrat; chiffrat = rot13(myString); return chiffrat; }

As this is not intended as an encryption 101, we should settle on a comparatively simple substitution cipher. ROT13 is not difficult, but it can be programmed quite verbosely. Dsoares\' implementation comes with a set of \'speaking\' variables and provides lots of food for our obfuscator.

function rot13(str) { var re = new RegExp("[a-z]", "i"); var min = \'A\'.charCodeAt(0); var max = \'Z\'.charCodeAt(0); var factor = 13; var result = ""; str = str.toUpperCase(); for (var i=0; i<str.length; i++) { result += (re.test(str[i]) ? String.fromCharCode((str.charCodeAt (i) - min + factor) % (max-min+1) + min) : str[i]); } return result; }

Performing an obfuscation run of the code is simple. Invoke javascript-obfuscator, and pass in a dot to tell the program to work on all files found in the current working directory. This shows the result on the author\'s IBM workstation:

tamhan@tamhan-thinkpad:~/FutureObfuscate/ code$ javascript-obfuscator .

Firing off the files directly into the container folder is inefficient, as names must be changed before they can be used. A better way involves the use of the output parameter. If javascript-obfuscator finds it, the program generates a subfolder in the current working directory and dumps the results of its labours there.

tamhan@tamhan-thinkpad:~/FutureObfuscate/code$ javascript-obfuscator . --output ./obfusca

Dive into the \'obfusca\' folder and open the new version of worker.js to feast your eyes on the abomination that is shown accompanying this step. The code\'s formatting was mangled badly. Method names, however, remained the same, as they are needed for external invocations. Furthermore, strings now sit in an array where an attacker can harvest them conveniently.

var a0_0xb9e2=[\'Hello\\x20from\\x20Future \\x20plc\',\'[a-z]\',\'test\',\'charCodeAt\'];

Javascript-obfuscator comes with a selection of string-mangling algorithms, which can be configured using --string-array-encoding. Keep in mind that the output directory must be emptied out before each invocation, because forgetting to do so leads to \'recursive\' obfuscation of the output files from the previous run.

tamhan@tamhan-thinkpad:~/FutureObfuscate/code$ javascript-obfuscator . --output ./obfusca --string-array-encoding base64

At this point, our obfuscator\'s output looks different the array at the top of the file now is much less readable. This, however, does not solve all problems. If you perform a few obfuscation cycles, you will eventually end up with mark-up similar to the one accompanying this step:

var a0_0x31e5=[\'bGVuZ3Ro\',\'dGVzdA==\',\'

ZnJvbUNoYXJDb2Rl\',\'Y2hhckNvZGVBdA==\',\' dG9VcHBlckNhc2U=\']; . . . . . var myString=\'Hello\\x20from\\x20Future\\x20plc\'

Obfuscators work in a pressure field between high performance and code protection. One way to address the problem involves \'randomising\' elements. The runtime-expensive obfuscations are not added to all nodes, but only to a subset. Detecting if nodes are affected or not is usually done via a random number generator, whose sensitivity can be tuned.

The above-mentioned random number generator emits numbers ranging from zero to one. If the number is larger than the threshold, the modification will not take place. Setting stringArrayThreshold to a value of one means that all numbers are smaller than the threshold, ensuring that each and every string gets mangled.

tamhan@tamhan-thinkpad:~/FutureObfuscate/code$ javascript-obfuscator . --output ./obfusca --string-array-encoding base64 --stringArrayThreshold 1

Analysis tools such as JSNice profit from having a small code base. Given that javascript-obfuscator breaks the code into an AST anyway, nothing speaks against inserting \'garbage code\' on the fly. As this feature adds significant bloat, developers must activate it manually via the two parameters shown next to this:

--dead-code-injection <boolean> --dead-code-injection-threshold <number>

Running the obfuscator with both --dead-code-injection true and --dead-code-injection-threshold 1 set leads to a filesize of about 2.5KB. Attempting to nicify the code leads to an almost indecypherable wall of mostly tautological JavaScript.

Breaking code into an AST allows for deep transformations. Setting the controlFlowFlattening attribute to true tells the program that it can de-inline function calls. This leads to a significant expansion of the generated code keep in mind that the results can take a 150 per cent performance hit.

Artefacts from debugging are the world\'s greatest gift to hackers. A few calls to console.log() and its friends can give an attacker valuable information on what happens inside the program a good example would be the snippet shown:

function doTheTrick() { var chiffrat; console.log("Preparing to encrypt chiffrat"); chiffrat = rot13(myString); console.log("Returning chiffrat"); return chiffrat; }

We can attempt to re-obfuscate the program with the command below. It disables the string caching feature and should disable console logging via redirection.

tamhan@tamhan-thinkpad:~/Desktop/DeadStuff/2018Nov/FutureObfuscate/code$ javascript-obfuscator . --output ./obfusca --string-array-encoding false --disableConsoleOutput true

Obfuscator and browser vendors fight a long and bitter battle about the debugger function. Due to this, \'aggressive\' measures, such as console redirection of the program flow interruptors shown accompanying this step, usually don\'t work for long.

debugProtection: false, debugProtectionInterval: false,

Installing the entire Node.js package for obfuscating one or two files is pointless. Visit obfuscator.io to access an online version of the program that lives in your browser. Check and comboboxes below the main input let you modify program behaviour as outlined in the steps above.

Click to see all the parameters you\'ll need

The developer team maintains relatively detailed documentation explaining the way the various command line parameters interact with one another. Simply visit head here if the \'short help\' output shown above does not help you reach your goal.

JS Nice is an easy way to render your code unreadable

The ETH Zurich provides a de-obfuscation service hosted at jsnice.org. It uses neural networks and a knowledge database made up of existing code to determine variable names. While sprucing up the formatting usually works pretty well, some of the names such as pixelSizeTargetMax in the encryption routine accompanying this step can be rather amusing.

function rot13(PL$120) { /** @type {!RegExp} */ var insufficientRegExp = new RegExp (a0_0x395d("0x1"), "i"); var pixelSizeTargetMax = "A" ["charCodeAt"](0); var zeroSizeMax = "Z"["charCodeAt"](0); /** @type {number} */ var minh = 13; /** @type {string} */ var out = "";

Generate, the award winning conference for web designers, returns to NYC on April 24-25! To book tickets visit www.generateconf.com

This article was originally published in issue 283 of creative web design magazine Web Designer. Buy issue 283 here or subscribe to Web Designer here.

Firebase authentication in Chrome Extension popup

I\'m trying to get authentication (email/password) to work in a Chrome Extension. I seems to work just fine if I put my authentication code in the background script. However I can\'t seem to get it to work as a browser action script.

I used the following code as the base of my extension: https://github.com/firebase/firebase-chrome-extension

Firebase.enableLogging(true); var f = new Firebase(\'https://myapp.firebaseio.com/\'); f.authWithPassword({ email : "a@b.cd", password : "1234" }, function(error, authData) { if (error) { alert("Login Failed!", error); } else { alert("Authenticated successfully with payload:", authData); } });

And I kept the existing browser_action.html unchanged:

<!doctype html> <style type="text/css"> #mainPopup { padding: 10px; height: 200px; width: 400px; font-family: Helvetica, Ubuntu, Arial, sans-serif; } h1 { font-size: 2em; } </style> <h1>Firebase test!</h1> Clicks: <script type="text/javascript" src="https://cdn.firebase.com/v0/firebase.js"></script> <script src="browser_action.js"></script>

When I load the extension and click on the icon it gives the following error in the console:

Uncaught TypeError: f.authWithPassword is not a function

If I move the firebase code and authentication code to background.js file it works and gives me an alert that it is succesfully authenticated.

What am I doing wrong or why is this not possible?'

Building a Chrome Extension with Firebase RTD

2018-04-03

Recently, I was tasked to prototype a solution that was most easily solved by building a native Chrome Extension alongside a companion web application. I needed a way for these two applications to share data amongst each other, and from a bit of research I learned that Chrome Extensions have an API for interacting with Chrome Storage, but interacting with the browser local storage was cumbersome. To bridge the data-gap between these two applications, I brought in Firebase Real-Time Database (RTD). We have been using it for several projects in the Originate portfolio and it has proven to be a very valuable tool. Firebase is essentially a backend-as-a-service that hosts your data and allows multiple applications to hook into it. While both were interesting, in this post, I will walk you through how to integrate Firebase RTD into a Chrome App.

Configuring Firebase for Chrome Extensions

If this is your first time creating a Chrome Extension, I recommend this awesome boilerplate. Hooking up Firebase to your Chrome Extension requires an added layer of complexity due to certain restraints on security as well as architecture best practices. Chrome Apps have a specific Content Security Policy (CSP) which prevents inline scripting and instantiates that policy to mitigate against cross-site scripting issues. There are more compliancy rules to consider generally, but for the purpose of Firebase configuration, you will need to specify that certain Google/Firebase routes are in the clear. To configure your Chrome App, a great place to start is the Authentication Boilerplate as a guideline for the following two preliminary steps:

import the Firebase library as an inline script in your HTML template.

update the content_security_policy value in your manifest.json file with the following:

{ ... "content_security_policy": "script-src \'self\' https://www.gstatic.com/ https://*.firebaseio.com https://www.googleapis.com; object-src \'self\'; connect-src \'self\' wss://*.firebaseio.com;", ... }

These rules let your Chrome Extension download the Firebase client library and connect to Firebases websocket server.

Note that the Firebase Authentication boilerplate has the configuration for

protecting your Extension with a login screen. It is a great reference if you are building more than a rapid prototype and are looking into protecting your database with proper ruling.

As a result of the way the library needs to be imported, your Firebase app (as well as the RTD event hooks) will need to be instantiated in the background script. Data changes will alter state housed in the Chrome Storage which is accessible to your background script, the popup render code, and in content scripts. Each operates within its own scope as background scripts run on Extension load, content scripts are JavaScript files that run in the context of web pages, and popup render code runs when the menu is opened. In order to access your Firebase database, you will need to utilize the Chrome Message API to relay changes to the RTD.

What does this look like in practice? Lets dig into some code:

// this file will run once on extension load var config = { apiKey: "[insert api key]", authDomain: "[insert auth domain]", databaseURL: "[insert database url]", projectId: "[insert project id]", storageBucket: "[insert storage bucket]", messagingSenderId: "[insert message sender id]" }; const app = firebase.initializeApp(config); const appDb = app.database().ref(); // instantiate global application state object for Chrome Storage and feed in firebase data // Chrome Storage will store our global state as a a JSON stringified value. const applicationState = { values: [] }; appDb.on(\'child_added\', snapshot => { applicationState.values.push({ id: snapshot.key, value: snapshot.val() }); updateState(applicationState); }); appDb.on(\'child_removed\', snapshot => { const childPosition = getChildIndex(applicationState, snapshot.key) if (childPosition === -1) return applicationState.values.splice(childPosition, 1); updateState(applicationState); }); appDb.on(\'child_changed\', snapshot => { const childPosition = getChildIndex(applicationState, snapshot.key) if (childPosition === -1) return applicationState.values[childPosition] = snapshot.val(); updateState(applicationState); }); // updateState is a function that writes the changes to Chrome Storage function updateState(applicationState) { chrome.storage.local.set({ state: JSON.stringify(applicationState) }); } // getChildIndex will return the matching element in the object function getChildIndex(appState, id) { return appState.values.findIndex(element => element.id == id) } // if your Chrome Extension requires any content scripts that will manipulate data, // add a message listener here to access appDb: chrome.runtime.onMessage.addListener((msg, sender, response) => { switch (msg.type) { case \'updateValue\': appDb.child(msg.opts.id).set({ value: msg.opts.value }); response(\'success\'); break; default: response(\'unknown request\'); break; } });

Thats it! All your extension needs to do is load from the Chrome local storage before rendering itself. If it or a content script needs to manipulate the RTD, the following snippet should be invoked:

chrome.runtime.sendMessage({type: \'updateValue\', opts: request.opts}, (response) => { if(response == \'success\') { // implement success action here } });

By following these steps, you should now have a Chrome Extension that will stay in sync with other companion applications! Now go celebrate the fruits of

your labor. Happy hacking!'

How North Korea uses bitcoin to get around US sanctions

By: Alex Ward | 2018-02-28

The US keeps imposing new sanctions on North Korea over its nuclear program but theyre not changing Kim Jong Uns behavior and for a surprising reason: Pyongyangs skill at using cryptocurrencies.

Five days ago, America sanctioned dozens of ships, shipping companies, and other firms that allegedly help Pyongyang fund its nuclear and missiles programs. That move is part of a broader US strategy to starve the North Korean regime of money it would use to improve its weapons. The Trump administration wants to make these punishments so severe that Pyongyang would potentially be open to negotiations over the programs.

The problem is North Korea can partially skirt financial restrictions with cryptocurrencies, an online form of money, according to Priscilla Moriuchi, formerly a top National Security Agency official charged with overseeing cyber threats from East Asia.

Moriuchi, who is now at the digital intelligence firm Recorded Future, estimates that North Korea earns between $15 million and $200 million by creating and selling cryptocurrencies and then turning it into hard cash. (Their take fluctuates depending on the digital currencys worth when North Korea cashes out.) Thats not enough money to fully fund North Koreas weapons programs, to be sure, but it ensures they dont completely shut down.

So even though sanctions hurt North Koreas physical economy, there are currently few ways America can curb Pyongyangs growing digital economy. That means North Korea has an even greater ability to keep its nuclear program running than we might think.

I would bet that these coins are being turned into something currency or physical goods that are supporting North Koreas nuclear and ballistic missile program, Moriuchi told me.

A transcript of our conversation, lightly edited for length and clarity, follows.

Why is cryptocurrency useful for North Korea?

There are not very many ID or security requirements, and many of these currencies are founded around or at least very much focused on security and anonymity for their users.

That kind of makes it a perfect platform for countries like North Korea, which

are pretty isolated from the international financial system to begin with, to use this new form of commerce thats thus far existed pretty much outside of countries regulatory control.

How does a country collect, or mine, bitcoin? Is it different from how an individual would do so?

Mining is just the way in which a cryptocurrency is generated.

Creating a single coin requires many computers to solve a number of mathematical problems. And once youve solved the problems, then it builds whats called a block. That block is essentially the currency itself you get rewards for creating a block.

Anyone can create a block, whether youre an individual or a country. It tends to be a computationally intensive and energy-intensive process. You need access to a very stable and high-bandwidth internet connection.

Then once you build the block you get the reward, which is typically a reward in that currency.

The US, along with the United Nations, continually sanction North Korea. They usually try to stop North Korea from trading their top exports, like iron, coal, and seafood.

But youre basically saying that North Korea is using digital currency in order to make money and partially evade those kinds of physical sanctions imposed on them, right?

The majority of international and US sanctions thus far have focused on pressure points on territorial North Korea and the North Korean regime. Those sanctions have been ongoing for years and years and years at this point, but the regime has not abandoned its drive to develop ballistic missiles or nuclear weapons.

North Korea, then, uses the internet to support itself, and its weapons programs, while getting around these sanctions.

I would bet that these coins are being turned into something currency or physical goods that are supporting North Koreas nuclear and ballistic missile program

Is North Korea doing all of this from home, or do they have operations elsewhere?

One of the things we know from defector testimony, for example, is that North Korea has built a network of operational bases in foreign countries where they execute cyber operations.

Theres some in China and some in Southeast Asia. Weve seen a good deal of North Korean activities in countries such as India, Malaysia, the Philippines, and we believe there are operational bases in some of those countries. We havent gotten all the way down yet to a street address or anything, though.

North Korea cyber operators sent to these foreign operations bases have two jobs. One, to conduct the operations that the North Korean regime tells them to conduct. And two, to earn money. The majority of that money, about 80 to 90 percent, is sent back to the regime, and only a small amount is for the operators themselves to keep.

How big is North Koreas cryptocurrency operation?

I saw a small-scale North Korean bitcoin mining operation last summer, but not anything on the factory-level scale like we will see sometimes in, like, China or Russia, for example.

On the surface, all of this sounds pretty sketchy and illegal.

But its not illegal in any way, unlike a lot of the things that North Korea is doing today to generate funds: counterfeiting US dollars and cigarettes or selling wildlife and illegal drugs.

Have we seen other countries use cryptocurrencies to buy physical things? We dont necessarily know that North Koreas buying fishing boats, but have we seen Russia buy stuff too?

That can happen: Any user can buy physical things with bitcoin.

You can buy pizzas at restaurants. You can buy goods at stores around the world. You can buy malware from dark web markets online. We havent seen anyone buying coal, for example, but just because we havent seen it doesnt mean its not out there.

But North Korea has such extensive criminal networks that have been well-established for decades to facilitate illegal activities. If Pyongyang were able to cash out into physical currency, it would be relatively easy for them to move that currency back into North Korea and to buy things with the physical currency.

How could North Korea could turn digital cash into hard cash?

There was a quite well-publicized banking theft from the Bank of Bangladesh back in 2015 that North Korea was responsible for. They ended up transferring, or stealing, about $81 million from the Central Bank of Bangladesh that then got transferred to bank accounts in the Philippines.

Those accounts were then cashed out within a few days, and then the money was run through casinos in the Philippines and Macau and basically disappeared.

Thats a great example of money going from the virtual to the physical.

The US has gotten quite good at sanctioning physical things. Why arent we so good at stopping digital currency activity?

The problem, especially when it comes to cryptocurrencies, is that theyre so loosely regulated. There are relatively few countries that impose any type of security on the network or identification laws for users of the exchanges. Theres not a document or a paper trail, so its very hard to find targets for any sort of sanctions.

We can say North Korea is using bitcoin, but we cant sanction Bitcoin as an entity because Bitcoin is not a person responsible its just distributed technology. So instead, were having to drill even deeper and try to look for wallets that North Korea has used or even exchanges that can be compelled to give data to law enforcement.

I think its just the lack of regulations in the digital world, kind of combined with such a broad target space, thats made it difficult to sanction.

Would you place a bet that North Korea is using digital currency in some way to improve its nuclear program?

I would bet that these coins are being turned into something currency or physical goods that are supporting North Koreas nuclear and ballistic missile program.

It could be pretty simple. After they cash it out, they take the hard cash to a bank that they work closely with. Once they deposit the cash in that bank, then it becomes part of their assets.

If you had a meeting with President Trump or UN Ambassador Nikki Haley to talk about how to stop North Korea in the digital currency space, what would you tell them?

First, we would need to enlist partner countries to increase regulations on the exchanges and the wallet providers. That helps create a paper trail we can use to identify North Korean accounts and how North Korea is moving these currencies.

And second, we should establish a robust sharing environment similar to the one that governments have with financial institutions, where you have to report or track transactions over a certain dollar amount, like $10,000. We should set some limits on when these exchanges have to report transactions over a certain number of, say, coins. That could be another way we could find some of this criminality as well.

How scared should we be that North Korea is going to continue getting better in the digital space, which therefore helps it to continue to get better in its nuclear and missiles program?

When it comes to consequences for cyber operations, there have been relatively few for any country. That holds true for North Korea as well.

But even so, there doesnt seem to be a resource constraint on North Korea right now. Theres certainly not an international constraint when it comes to limiting their movement in other countries or limiting their information technology or computer science training.'

Who is Vinay Dubey, the man that threatened a foot march from Mumbai to North India if train services weren't resumed?

The rumours that originally started as a WhatsApp forward eventually became mainstream media news when Marathi channel ABP Majha made it into a story. This resulted in contractors telling the migrant workers that services were resumed.

The man behind this problem was Vinay Dubey, a self-proclaimed entrepreneur and social worker based out of Navi Mumbai. Dubey has been formally arrested by Mumbai police now, under IPC sections 117, 153 A, 188, 269, 270, 505 (2) and Section 3 of The Epidemic Diseases Act. He will be produced before a local court later today.

His arrest comes after uploading a video on YouTube that threatened a foot march towards North India if train services werent resumed.

But the question arises: Who is Vinay Dubey? According to his Facebook profile, Dubey seems to have quite a fan following, with 2,18,573 followers on his site. He had even fought elections in 2019 Lok Sabha as an independent candidate

On April 13, Dubey put out a Facebook post where he warned of consequences if the Central and state governments did not resume train services to let migrant workers go back home. He had shared other posts where he claimed that he had arranged 40 buses for stranded migrant workers stuck in Maharashtra, so that they could be taken back homes.

A day earlier, Dubey shared a video where he was critical of Prime Minister Narendra Modis implementation of the lockdown, stating that it had been executed without any planning or keeping the poor in mind. Thousands of migrant workers are stuck without any money in their pocket or a chance to go back home, Dubey said.'